

BhuNexis

AI-Powered Land Record Digitization & Validation Platform

Master Technical Project Documentation • Source of Truth Reference • Generated: September 19, 2026

SIH 2026 Hackathon

Python 3.13 + FastAPI

React 18 + Vite + Leaflet

PostgreSQL 15 + PostGIS

MinIO S3 Storage

1. Executive Summary & Problem Statement

BhuNexis is an enterprise-grade government land records digitization and validation platform designed to solve systemic challenges in legacy land administration systems. Historical land revenue records (such as Khata, Khasra, Jamabandi, and Mutation register documents) primarily exist as unindexed scanned PDFs, physical paper archives, handwritten records, and low-resolution images containing complex tabular data.

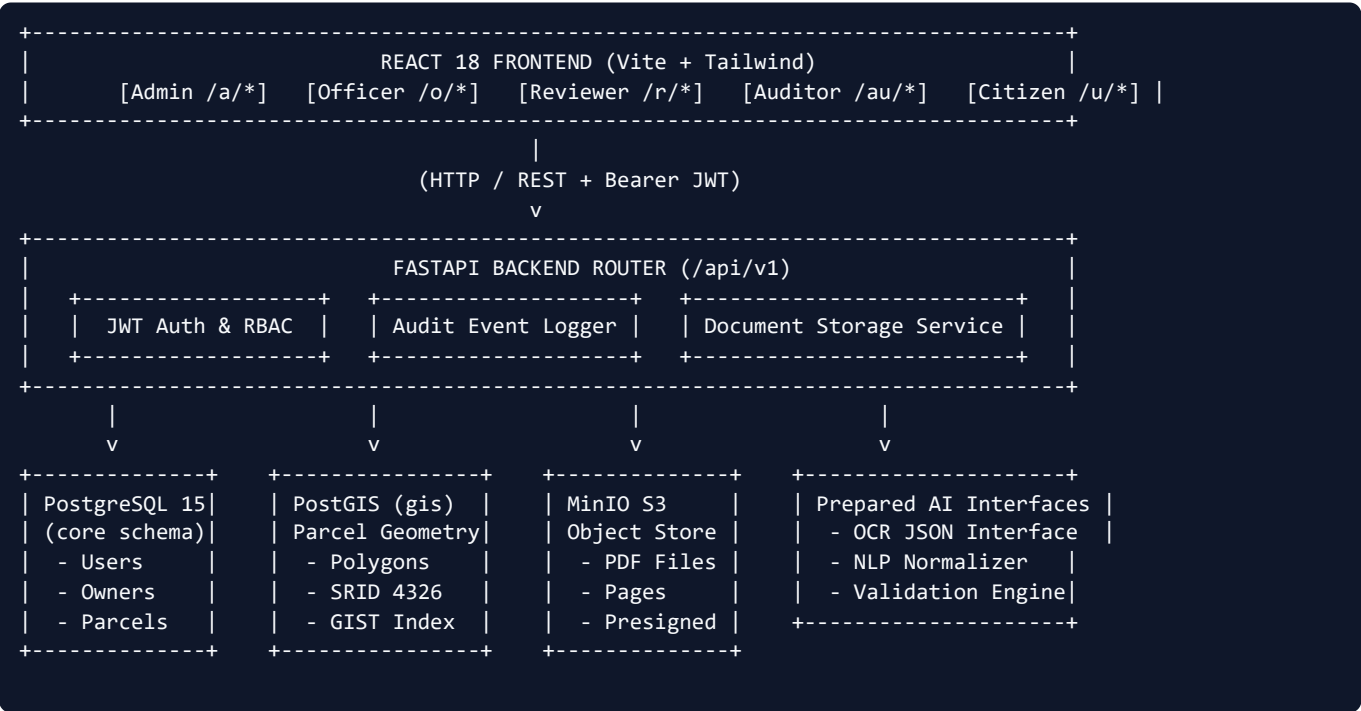
Core Objective: Transform unstructured, unvalidated physical and scanned land records into structured, AI-extracted, rule-validated, spatially cross-referenced, and fully auditable digital land records backed by PostGIS spatial boundaries and human-in-the-loop review queues.

End-to-End Processing Pipeline:

Document Upload (PDF/Image) → MinIO Storage → Preprocessing & Page Extraction → OCR Text & Bounding Box Detection → NLP Entity Extraction & Normalization → Multi-Rule Validation Engine → GIS Spatial Cross-Reference → Human-in-the-Loop Review Queue → Final Record Verification → PostgreSQL / PostGIS Spatial Persistence → Citizen & GIS Access

2. High-Level Architecture Overview

BhuNexis follows a modern, decoupled micro-architecture separating presentation, core API orchestration, object storage, spatial data persistence, and specialized AI processing engines.



3. Complete Repository Directory Structure

```

BhuNexis/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── deps.py                # FastAPI Dependency Injection (Auth, DB, RBAC)
│   │   │   └── v1/
│   │   │       ├── api.py             # Central API Router Aggregator
│   │   │       └── endpoints/
│   │   │           ├── audit.py        # Audit Event Analytics & Logs API
│   │   │           ├── auth.py         # Login, Register, Refresh Token API
│   │   │           ├── citizen.py      # Citizen Parcel Search & Verification API
│   │   │           ├── dashboard.py    # Role Dashboards (Admin, Officer, Reviewer, Auditor, Citizen)
│   │   │           ├── documents.py    # Document Upload, Status, & File Download API
│   │   │           ├── health.py       # System Health Check API
│   │   │           ├── integrations.py # OCR/NLP External Webhook Listeners
│   │   │           ├── map.py          # GET /map/parcels PostGIS GeoJSON Spatial API
│   │   │           ├── parcels.py      # Parcel CRUD & Owner Linkage API
│   │   │           ├── profile.py      # User Profile Management API
│   │   │           ├── reviews.py     # Human Review Queue & Field Correction API
│   │   │           ├── settings.py     # System Configuration & OCR Threshold API
│   │   │           ├── users.py        # Admin User Management API
│   │   │           └── validations.py  # Rule Validation Trigger & Results API
│   │   ├── core/
│   │   │   ├── audit.py               # Audit Logging Dispatcher
│   │   │   ├── config.py              # Pydantic Settings & Environment Variables
│   │   │   ├── rbac.py                # AppRole Enum & Permissions Matrix
│   │   │   ├── security.py            # Bcrypt Password Hashing & JWT Token Utility
│   │   │   └── storage.py             # MinIO Object Storage Manager
│   │   ├── db/
│   │   │   ├── base.py                # SQLAlchemy DeclarativeBase
│   │   │   └── session.py             # DB Engine & Session Maker
│   │   ├── models/
│   │   │   └── all_models.py          # Unified SQLAlchemy DB Models (14 Tables)
│   │   ├── schemas/
│   │   │   ├── auth.py               # Login & Token Pydantic Schemas
│   │   │   ├── document.py           # Document Upload & Extracted Field Schemas
│   │   │   ├── parcel.py             # Parcel & GeoJSON Feature Schemas
│   │   │   └── review.py              # Review Case & Correction Schemas
│   │   └── main.py                   # FastAPI Application Initialization & CORS
│   ├── tests/
│   │   ├── test_api.py               # API Endpoint Integration Test Suite
│   │   └── test_health.py            # Health Check Test Suite
│   ├── pyproject.toml                # Python Dependencies & Package Metadata
│   └── pytest.ini                    # Pytest Runner Configuration
├── database/                        # Prepared Schema Migration & SQL Seeds
├── docs/                            # Project Technical Documentation PDFs
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── common/               # StatusBadge, AccessDenied, Navbar, Footer
│   │   │   ├── layout/               # Sidebar & Main App Header Controls
│   │   │   └── map/
│   │   │       └── MapViewer.jsx      # Cadastral GIS Map (Leaflet + AutoFit + Plot Labels)
│   │   ├── pages/
│   │   │   ├── Home.jsx              # Platform Overview & Workflow Landing Page
│   │   │   ├── Login.jsx             # Role-Based Authentication Interface
│   │   │   ├── MapViewPage.jsx       # Fullscreen Cadastral GIS Map Page
│   │   │   ├── ProfilePage.jsx       # User Profile & Security Settings
│   │   │   ├── SignUp.jsx            # Citizen Self-Registration Page
│   │   │   ├── UploadDocumentPage.jsx # Officer Bulk Document Upload Interface
│   │   │   ├── admin/
│   │   │   │   ├── UserManagement.jsx # UserManagement.jsx, SystemSettings.jsx
│   │   │   │   └── AuditLogsPage.jsx  # AuditLogsPage.jsx
│   │   │   ├── auditor/
│   │   │   │   └── CitizenSearch.jsx  # CitizenSearch.jsx
│   │   │   ├── dashboards/
│   │   │   │   ├── Admin, Officer, Reviewer, Auditor, Citizen Dashboards
│   │   │   │   └── HumanReviewPage.jsx # HumanReviewPage.jsx
│   │   └── routes/
│   │       └── AppRoutes.jsx          # React Router v6 Route Definitions

```

```
├── ProtectedRoute.jsx      # Role Guard Route Wrapper
├── services/
│   ├── apiClient.js       # Axios Instance with JWT Interceptor
│   ├── authService.js     # Auth API Service
│   └── mapService.js      # Cadastral GIS Map API Service
├── App.jsx                # Application Root Component
├── main.jsx               # Vite React Entrypoint
├── package.json           # Frontend NPM Dependencies
├── vite.config.js         # Vite Server & Backend Proxy Settings
├── nlp/                   # Prepared NLP Entity Extraction Module Interface
├── ocr/                   # Prepared OCR Processing Pipeline Interface
└── validation/           # Prepared Multi-Rule Validation Engine Interface
```

4. Frontend Architecture & Page Routing Flow

The frontend is built with React 18, Vite, and TailwindCSS using a Role-Based Access Control routing model.

Role-Based Route Table:

Role	Dashboard Route	Feature Routes	Access Permissions
ADMIN	/a/dashboard	/a/users , /a/settings , /map , /profile	Full system administrative access, user management, global settings, audit logs.
OFFICER	/o/dashboard	/o/upload , /map , /profile	Document upload, batch ingestion, document status monitoring.
REVIEWER	/r/dashboard	/r/review , /map , /profile	Human review queue, side-by-side OCR field correction, approve/reject land cases.
AUDITOR	/au/dashboard	/au/audit , /map , /profile	Read-only compliance audit history, system analytics, event timeline tracking.
CITIZEN	/u/dashboard	/u/search , /map , /profile	Read-only verified parcel search, land ownership records lookup, GIS map view.

5. Role-Based Access Control (RBAC) Specification

BhuNexis enforces RBAC at both the frontend React routing layer (via ProtectedRoute.jsx) and backend FastAPI dependency layer (via require_roles() in deps.py).

Capability	ADMIN	OFFICER	REVIEWER	AUDITOR	CITIZEN
Upload Land Documents	YES	YES	NO	NO	NO
Human Review & Field Edit	YES	NO	YES	NO	NO
View Audit Logs & Analytics	YES	NO	NO	YES	NO
User Management & Settings	YES	NO	NO	NO	NO
Search Verified Records & GIS Map	YES	YES	YES	YES	YES

6. Backend API Architecture & Endpoints

The backend is built using FastAPI (Python 3.13) with SQLAlchemy 2.0 ORM, PostgreSQL + PostGIS, and MinIO S3 Object Storage.

Implemented FastAPI Endpoint Registry:

Method	Endpoint	Purpose	Allowed Roles	Status
POST	<code>/api/v1/auth/login</code>	Authenticate user & issue JWT Bearer Token	Public	IMPLEMENTED
POST	<code>/api/v1/auth/register</code>	Register citizen account	Public	IMPLEMENTED
GET	<code>/api/v1/auth/me</code>	Get current authenticated user profile	All Roles	IMPLEMENTED
POST	<code>/api/v1/documents/upload</code>	Upload document PDF/Image to MinIO & record in DB	OFFICER, ADMIN	IMPLEMENTED
GET	<code>/api/v1/documents</code>	List uploaded documents with status filters	OFFICER, ADMIN, REVIEWER	IMPLEMENTED
GET	<code>/api/v1/map/parcels</code>	Query PostGIS spatial parcels as GeoJSON FeatureCollection	All Roles	IMPLEMENTED
GET	<code>/api/v1/reviews</code>	Fetch human review queue items	REVIEWER, ADMIN	IMPLEMENTED
PATCH	<code>/api/v1/reviews/{id}</code>	Submit corrected OCR field value	REVIEWER	IMPLEMENTED
POST	<code>/api/v1/reviews/{id}/approve</code>	Approve review case & mark verified	REVIEWER	IMPLEMENTED
POST	<code>/api/v1/reviews/{id}/reject</code>	Reject review case	REVIEWER	IMPLEMENTED
GET	<code>/api/v1/dashboard/citizen</code>	Get citizen dashboard land records summary	CITIZEN, ADMIN	IMPLEMENTED
GET	<code>/api/v1/audit/logs</code>	Fetch system audit trail logs & analytics	AUDITOR, ADMIN	IMPLEMENTED
POST	<code>/api/v1/integrations/ocr-webhook</code>	Receive OCR/NLP service extraction payload	System Integration	PREPARED

7. Complete Database Schema (PostgreSQL + PostGIS)

The database `bhunexis` utilizes PostgreSQL 15 with two schemas: `core` for structured business entities and `gis` for PostGIS spatial geometries.

Database Table Definitions (14 Tables):

Table Name	Schema	Primary Columns & Foreign Keys	Description
<code>users</code>	core	<code>id, email, password_hash, full_name, role</code>	User credentials and RBAC authorization roles.
<code>owners</code>	core	<code>id, owner_uid, full_name, father_name, village, district</code>	Landowner registry profiles.
<code>parcels</code>	core	<code>id, parcel_uid, survey_number, khasra_number, khata_number,</code>	Land parcel record registry.

Table Name	Schema	Primary Columns & Foreign Keys	Description
		<code>plot_number, area, status</code>	
<code>land_rights</code>	core	<code>id, owner_id (FK), parcel_id (FK), ownership_type, share_percentage</code>	Ownership share linkage mapping owners to parcels.
<code>parcel_geometry</code>	gis	<code>id, parcel_id (FK), geometry (Polygon, 4326), gis_area_sq_m</code>	PostGIS spatial geometry vector polygon boundaries.
<code>documents</code>	core	<code>id, document_uid, title, file_path, file_type, ocr_status, uploaded_by (FK)</code>	Uploaded land record metadata and MinIO storage paths.
<code>document_pages</code>	core	<code>id, document_id (FK), page_number, file_path</code>	Individual page images extracted from PDF documents.
<code>extracted_fields</code>	core	<code>id, document_page_id (FK), field_name, extracted_value, confidence_score, validation_status</code>	OCR & NLP extracted land attributes with confidence scores.
<code>validation_results</code>	core	<code>id, parcel_id (FK), document_id (FK), validation_type, status, severity, message</code>	Rule validation execution outputs.
<code>review_cases</code>	core	<code>id, parcel_id (FK), document_id (FK), assigned_to (FK), priority, status, reviewer_comment</code>	Human review queue items for low-confidence or conflicting data.
<code>audit_events</code>	core	<code>id, user_id (FK), action, entity_type, entity_id, old_value, new_value, created_at</code>	Immutable system audit trail tracking all actions.
<code>mutations</code>	core	<code>id, parcel_id (FK), seller_id (FK), buyer_id (FK), status</code>	Land mutation and transfer history.
<code>registrations</code>	core	<code>id, parcel_id (FK), deed_number, registration_date</code>	Official deed registration records.
<code>evidence</code>	core	<code>id, review_id (FK), file_path, file_type</code>	Supporting evidence attachments uploaded by reviewers.

8. GIS & Cadastral Mapping Subsystem

The Cadastral Mapping subsystem provides visual spatial mapping of land boundaries directly linked to PostGIS database records.

- **PostGIS Spatial Query:** Endpoint `GET /api/v1/map/parcels` executes `ST_AsGeoJSON(pg.geometry)::json` to query spatial polygons and populates GeoJSON properties with plot number, survey number, khasra number, area, and status.
- **Frontend Map Rendering (`MapView.jsx`):**
 - **Line Boundaries & Styling:** Renders polygon borders with high-contrast color coding (Emerald stroke for Verified, Crimson stroke for Review Required, Amber stroke for Draft).
 - **Plot Number Badges:** Displays permanent plot number badges (e.g., `PLT-04011`) centered on top of each parcel boundary polygon using Leaflet tooltips.

- **Auto-Fit View Bounds:** Includes an `AutoFitBounds` sub-component calling `map.fitBounds()` to automatically zoom and center the map on loaded PostGIS parcels.
- **Multi-Layer Basemaps:** Toggles between Standard OpenStreetMap, Esri World Imagery Satellite, and Dark GIS canvas.

9. Comprehensive Component Status Matrix

Component	Status	Details & Implementation Notes
Frontend React App	IMPLEMENTED	Vite, TailwindCSS, custom themes, responsive sidebar/header layouts.
Role Dashboards	IMPLEMENTED	Dashboards for Admin, Officer, Reviewer, Auditor, and Citizen.
FastAPI REST Engine	IMPLEMENTED	Async routing, request validation, error handlers, CORS middleware.
JWT Auth & RBAC	IMPLEMENTED	Bcrypt hashing, JWT Bearer tokens, frontend & backend route guards.
PostgreSQL Database	IMPLEMENTED	14 tables across <code>core</code> and <code>gis</code> schemas with relationships.
PostGIS Spatial Geometry	IMPLEMENTED	Polygon geometries, spatial queries, GIST spatial index.
Cadastral Map Viewer	IMPLEMENTED	Leaflet vector lines, plot badges, auto-fit bounds, layer switcher.
MinIO Object Storage	IMPLEMENTED	Local storage fallback & MinIO client integration for PDF uploads.
Audit Trail System	IMPLEMENTED	Audit logging helper recording user action, entity ID, and diffs.
OCR Processing Engine	PREPARED INTERFACE	Backend integration endpoints ready; PaddleOCR integration prepared.
NLP Extraction Engine	PREPARED INTERFACE	Entity normalization contracts ready; transformer models planned.
Multi-Rule Validation	PREPARED INTERFACE	Rule validation data structures ready; automated rules planned.

10. Technical Glossary

- **PostGIS:** Spatial database extender for PostgreSQL object-relational database, adding support for geographic objects.
- **Cadastral Map:** A comprehensive map showing parcel boundaries, plot numbers, ownership details, and spatial dimensions.
- **GeoJSON:** An open standard format designed for representing simple geographical features along with non-spatial attributes.
- **Khasra / Khata / Survey Number:** Standard land record parcel identifiers used across Indian land revenue administration.
- **MinIO:** High-performance, S3-compatible object storage server used for storing raw scanned PDF documents and page renders.
- **RBAC (Role-Based Access Control):** An approach to restricting system access to authorized users based on defined roles.

11. AI Development Handoff Context

Project Context Summary for Future AI Coding Assistants:

BhuNexis is a full-stack land records digitization platform built using Python 3.13, FastAPI, SQLAlchemy 2.0, PostgreSQL 15 + PostGIS, MinIO, React 18, Vite, TailwindCSS, and Leaflet. The PostgreSQL database name is `bhunexis` containing 14 tables across `core` and `gis` schemas.

Strict Guidelines for Future AI Assistants:

1. **Do NOT recreate or drop the database:** Use the existing `bhunexis` PostgreSQL database.
2. **Preserve Schema Isolation:** Maintain the `core` schema for business entities and `gis` schema for spatial geometries.
3. **Preserve Database Relationships:** Maintain existing primary keys, foreign keys, and model properties in `all_models.py`.
4. **Security First:** Keep secrets and credentials in environment variables (`.env`). Never expose MinIO credentials or DB passwords to the frontend.
5. **Honest Status Reporting:** Do NOT claim OCR/NLP ML models are fully integrated inside the repository when they are prepared integration interfaces.